# 实验报告：磁盘调度模拟

## 1. 程序中使用的数据结构及符号说明

| 符号/数据结构 | 类型 | 说明 |
| --- | --- | --- |
| `vector<int>` | 动态数组 | 存储磁道请求序列及排序后的请求列表 |
| `vector<bool>` | 布尔数组 | 标记 SSTF 算法中已访问的请求 |
| `int algo` | 整数 | 算法选项：1=FCFS，2=SSTF，3=SCAN，4=C-SCAN |
| `int headPos` | 整数 | 当前磁头位置 |
| `int direction` | 整数 | 移动方向：1=向磁道号增大方向，0=向减小方向 |
| `vector<int> order` | 动态数组 | 记录磁头访问顺序（包含起始位置） |
| `long long totalDistance` | 长整数 | 计算并累加磁头移动总距离 |

## 2. 各种调度算法的处理流程及重要模块设计

### 2.1 先来先服务（FCFS）

- **流程**：按请求序列原始顺序依次访问。
- **模块设计**：
  - **输入解析模块**：读取并解析请求列表。
  - **访问顺序构建模块**：遍历请求列表，将每个请求追加到 `order` 。
  - **距离计算模块**：遍历 `order` 计算相邻两点距离并累加。

### 2.2 最短寻道优先（SSTF）

- **流程**：每次选择与当前磁头最近的未访问请求。
- **模块设计**：
  - **未访问标记数组**（`used`）：跟踪请求是否已访问。
  - **距离计算子函数**：比较当前 `headPos` 与各未访问请求的绝对差，选最小。
  - **迭代访问**：更新 `curr` 为刚访问的请求，标记为已访问，追加至 `order` 。

### 2.3 扫描法（SCAN）

- **流程**：磁头沿指定方向移动，访问该方向所有请求，然后折返访问反方向请求。
- **模块设计**：
  - **排序模块**：对请求序列升序排序，分割为小于 `headPos` （ `left` ）和大于等于 `headPos` （ `right` ）两组。

- **访问顺序生成器**：
  - 若方向为增大：先访问 `right` （升序），再访问 `left` （降序）；
  - 若方向为减小：先访问 `left` （降序），再访问 `right` （升序）。

## 2.4 循环扫描法（C-SCAN）

- **流程**：磁头沿指定方向移动，访问所有请求，到达末端后跳转至另一端继续。
- **模块设计**：
  - **排序与分组**：同 SCAN。
  - **访问策略**：
    - 增大方向：访问 `right` （升序），跳至最小后访问 `left` （升序）；
    - 减小方向：访问 `left` （降序），跳至最大后访问 `right` （降序）。

# 3. 源程序及注释

```
#include <bits/stdc++.h>
using namespace std;

// 解析逗号分隔的磁道请求序列
vector<int> parse_sequence(const string &s) {
    vector<int> seq;
    string num;
    for (char c : s) {
        if (c == ',') {
            if (!num.empty()) {
                seq.push_back(stoi(num));
                num.clear();
            }
        } else if (!isspace(c)) {
            num.push_back(c);
        }
    }
    if (!num.empty()) {
        seq.push_back(stoi(num));
    }
    return seq;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int algo;           // 1=FCFS, 2=SSTF, 3=SCAN, 4=C-SCAN
    int headPos;        // 当前磁头位置
    int direction;      // 1=向号增大方向, 0=向号减小方向
    string seqLine;

    // 读入
    if (!(cin >> algo)) return 0;
    cin >> headPos;
    cin >> direction;
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    getline(cin, seqLine);

    vector<int> requests = parse_sequence(seqLine);
    int n = requests.size();

    vector<int> order;  // 最终的磁头访问顺序 (包含起始 headPos)
    order.reserve(n + 1);
```

```
order.push_back(headPos);

if (algo == 1) {
    // 1. FCFS（先来先服务）
    for (int r : requests) {
        order.push_back(r);
    }

} else if (algo == 2) {
    // 2. SSTF（最短寻道时间优先）
    vector<bool> used(n, false);
    int curr = headPos;
    for (int k = 0; k < n; k++) {
        int bestIdx = -1;
        int bestDist = INT_MAX;
        for (int i = 0; i < n; i++) {
            if (!used[i]) {
                int d = abs(requests[i] - curr);
                if (d < bestDist) {
                    bestDist = d;
                    bestIdx = i;
                }
            }
        }
        used[bestIdx] = true;
        curr = requests[bestIdx];
        order.push_back(curr);
    }

} else if (algo == 3 || algo == 4) {
    // 3. SCAN 或 4. C-SCAN
    // 先对请求做排序
    vector<int> reqs = requests;
    sort(reqs.begin(), reqs.end());

    // 把请求分为两部分：小于 headPos 与 大于等于 headPos
    vector<int> left, right;
    for (int r : reqs) {
        if (r < headPos) left.push_back(r);
        else right.push_back(r);
    }

    if (algo == 3) {
        // SCAN
        if (direction == 1) {
            // 先向号增大方向：访问 right 按升序
```

```
            for (int r : right) order.push_back(r);
            // 然后向号减小方向：访问 left 按降序
            for (int i = (int)left.size() - 1; i >= 0; i--) {
                order.push_back(left[i]);
            }
        } else {
            // direction == 0，先向号减小方向：访问 left 按降序
            for (int i = (int)left.size() - 1; i >= 0; i--) {
                order.push_back(left[i]);
            }
            // 再向号增大方向：访问 right 按升序
            for (int r : right) order.push_back(r);
        }
    } else {
        // C-SCAN
        if (direction == 1) {
            // 先向号增大：访问 right 升序
            for (int r : right) order.push_back(r);
            // 到达最大后，跳到最小（wrap）：访问 left 升序
            for (int r : left) order.push_back(r);
        } else {
            // direction == 0，先向号减小：访问 left 降序
            for (int i = (int)left.size() - 1; i >= 0; i--) {
                order.push_back(left[i]);
            }
            // 到达最小后，跳到最大（wrap）：访问 right 降序
            for (int i = (int)right.size() - 1; i >= 0; i--) {
                order.push_back(right[i]);
            }
        }
    }
} else {
    // 无效选项
    return 0;
}

// 计算总移动距离
long long totalDistance = 0;
for (int i = 1; i < (int)order.size(); i++) {
    totalDistance += llabs(order[i] - order[i - 1]);
}

// 输出访问路径
for (int i = 0; i < (int)order.size(); i++) {
    cout << order[i];
    if (i + 1 < (int)order.size()) cout << ",";
```

```
    }
    cout << "\n";

    // 输出总移动距离
    cout << totalDistance << "\n";

    return 0;
}
```

## 4. 测试方法及结果分析

- **测试方法**：
  i. **随机生成**：在磁道范围 [0, 199] 内随机生成 10-20 个请求，测试四种算法。
  ii. **边界测试**：首尾请求均位于最小/最大磁道，验证 SCAN/C-SCAN 跳转正确性。
  iii. **方向切换测试**：不同方向输入，验证处理分支。
- **结果分析**：
  - FCFS 易造成“抖动”现象，整体寻道距离较大。
  - SSTF 能较好减少总距离，但可能导致“饥饿”现象。
  - SCAN 在重负载时性能稳定，但首尾反转带来额外开销。
  - C-SCAN 平均响应时间优于 SCAN，但总距离稍增。
- **错误分析**：
  - 测试中发现边界条件下未正确清空缓存，导致跳转逻辑错误，已通过重置 `left/right` 后修正。
  - SSTF 在所有请求相同距离时会随机选取，需指定规则以保证可预测性。

## 5. 实验经验及体会

此次实验通过独立设计并实现多种磁盘调度算法，加深了对操作系统磁盘管理策略的理解。实践中体会到：

1. **算法权衡**：不同策略在响应时间和公平性上各有优劣，需要根据 I/O 特征选择。
2. **边界处理**：边界条件测试至关重要，尤其针对循环调度需要额外关注跳转逻辑。
3. **代码组织**：模块化设计与注释帮助快速定位问题，便于后续扩展与维护。
4. **测试方法**：系统化测试用例设计能有效发现隐藏缺陷。

通过本次实验，不仅掌握了磁盘调度算法的核心思想，也提升了 C++ 编程及调试能力。